Patient-Independent TinyML ECG Classification: Generalization, Robustness, Quantization, and ESP32-S3 Deployment Trade-offs

1st Given Name Surname
Dept. name of organization
Name of organization
City, Country
email address or ORCID

2nd Given Name Surname
Dept. name of organization
Name of organization
City, Country
email address or ORCID

3rd Given Name Surname
Dept. name of organization
Name of organization
City, Country
email address or ORCID

4th Given Name Surname
Dept. name of organization
Name of organization
City, Country
email address or ORCID

***Abstract*—Low-cost wearable ECG screening on sub-$15 microcontrollers must balance accuracy, size and latency under tight memory and quantized arithmetic. We evaluate four compact architectures (1D-CNN, LSTM, GRU, TCN) for 3-class beat classification (Normal/APB/PVC) on MIT-BIH under identical patient-level folds. Unlike studies optimized for offline accuracy, we prioritize deployment-valid performance: patient-independent grouped CV (5×2), noise robustness (SNR 0–40 dB, 5 artifacts $\times$ 3 front-ends), partial external validation on SVDB (N/V only), paired FP32→INT8 quantization, and measured ESP32-S3 execution. Grouped-CV macro is 0.619 (CNN) / 0.615 (TCN) with APB as the bottleneck. Paired INT8 degrades macro by 0.21–0.24 with 33–40% prediction disagreement on identical folds; the evaluated LSTM/GRU graphs remain float32 through the tested converter path. On ESP-NN kernels, CNN (189 ms) / TCN (298 ms, incl. ~ 5 ms Butterworth) meet the 500-ms streaming hop (RTF 0.38/0.60) once the 19 KB/315 ms learned denoiser is replaced by the filter; float32 LSTM/GRU exceed the budget. Optimized kernel implementation materially changes the architecture–latency trade-off and can determine whether an otherwise compact model satisfies the streaming deadline.**

I. Introduction

Abnormal heart rhythms such as atrial premature beats (APB) and premature ventricular contractions (PVC) are common, often asymptomatic, and frequently go undetected until a serious event occurs. Low-cost single-lead ECG screening devices could make routine rhythm monitoring accessible in low-resource settings, but the economics of hardware gate the practicality: an ESP32-class microcontroller costs a few dollars, yet executes deep learning models under tight memory and compute constraints and with quantized arithmetic.

Recent work demonstrates ECG classification with convolutional networks on embedded targets, but most studies report only offline accuracy, leaving three gaps: (i) architectures are rarely compared on the same data pipeline, (ii) quantization effects and on-device latency are often estimated rather than measured, and (iii) on-device inference is rarely verified on silicon after quantization. Unlike studies optimized primarily for offline classification accuracy, this work prioritizes deployment-valid performance under patient-independent evaluation, quantization, noise corruption, and measured hardware constraints. This work does not propose a new ECG neural architecture. Instead, it provides a controlled deployment-oriented evaluation of four compact architecture families under identical patient-disjoint ECG folds, followed by noise stress testing, partial external transfer, paired post-training quantization, confidence calibration, and measured ESP32-S3 execution. The resulting analysis exposes deployment effects that offline accuracy alone cannot reveal, including minority-class instability, substantial PTQ degradation, preprocessing-driven deadline violations, and kernel-dependent changes in architecture–latency ranking.

II. Related Work

Beat-level ECG classification on MIT-BIH is a mature problem; convolutional and recurrent models report high per-class F1 for Normal and PVC classes, with atrial beats typically the most difficult minority class [1], [2]. For deployment, TensorFlow Lite Micro enables int8 inference on microcontrollers [3], and prior ESP32 ECG studies demonstrate the feasibility of a single CNN on this class of hardware [4]. Rhythm-level atrial fibrillation detection is conventionally addressed on longer windows with dedicated AF databases [5]; beat-level and rhythm-level tasks are deliberately kept separate in this work, and rhythm-level AF detection is not evaluated here. What is less established is a controlled comparison of architectures on one data pipeline with int8 quantization impact and hardware-measured latency. This paper provides that comparison and closes with an on-device feasibility verification of the quantized pipeline.

Convolutional and recurrent approaches to heartbeat classification now report strong results on MIT-BIH, from patient-specific 1-D CNNs [6] to cardiologist-level recurrent and convolutional models trained on large ambulatory corpora [7], [8] and on 12-lead ECG [9]; surveys of the area [10], [11], [12] and of the monitoring-system design space [13] consistently note that the atrial (minority) class remains the main difficulty. Most of these systems are evaluated offline on the full recording, leaving the embedded deployment question of quantized weights, int8 arithmetic, memory, and latency

on silicon only partially answered. Recent TinyML/embedded ECG deployments on MCU-class hardware (2022–2024) illustrate similar tensions but rarely provide the full combination of identical patient-disjoint folds, artifact stress tests, partial external transfer, PTQ, calibration, preprocessing ablation, and measured kernel-dependent execution pursued here.

For edge deployment the relevant tooling is now mature: integer-arithmetic-only quantization [14], [15] makes int8 inference practical on microcontroller-class parts, TensorFlow Lite Micro provides the runtime [3], TinyML design guidance targets the flash and memory budgets we report [16], and calibrated probabilities matter for thresholded decisions [17], [18]. The ESP32-S3's reference documentation defines the ADC, DMA, and low-power modes the firmware relies on [19].

TABLE I
PRIOR-WORK GAP

Work	Pat.-ind	Ext.	Noise	Quant	MCU	Calib	Latency
Kiranyaz [6]	NR	NR	NR	NR	NR	NR	NR
Hannun [7]	NR	✓	NR	NR	NR	NR	NR
Davidson [3]	NR	NR	NR	✓	✓	NR	✓
ESP32 ECG [4]	NR	NR	✓	✓	✓	NR	est
This work	✓	✓	✓	✓	✓	✓	✓

III. METHODS

A. Data and labels

Beat-level training uses the MIT-BIH Arrhythmia Database [1], [20] (48 recordings, 360 Hz, MLII lead), part of the public PhysioNet archive [21]. Each record is treated as one patient identity and all windows from a record are restricted to a single fold ($\text{Patients}_{\text{train}} \cap \text{Patients}_{\text{test}} = \emptyset$). We extract 1-second windows (360 samples) centered on R-peaks with three classes: Normal (N), Atrial Premature Beat (A), and Premature Ventricular Contraction (V); other annotations are discarded. Rhythm-level AF detection is scoped out. All segments are per-window normalized to [-1, 1] (subtract mean, divide by max absolute deviation), matching firmware. Figure 1 outlines the methodology.

$$\hat{x}_i = \frac{x_i - \mu}{\max_j |x_j - \mu|}, \quad \mu = \frac{1}{L}\sum_{i=1}^{L} x_i \tag{1}$$

B. Signal quality gate

A classifier can emit high-confidence predictions on corrupted input. We evaluate a heuristic signal-quality gate (10-s buffer rejected if near-flat, $> 5\%$ saturated, or high-frequency dominated) as an experimental ablation; it is disabled in the recommended deployment (see §V; SQI details in Supplementary).

$$\text{range} = \max_i x_i - \min_i x_i, \qquad \text{ratio} = \frac{\sum_{i=1}^{L-1}(x_{i+1} - x_i)^2}{\sum_{i=1}^{L}(x_i - \bar{x})^2} \tag{2}$$

The ratio is an inexpensive proxy for high-frequency corruption such as muscle artifact or baseline wander [22].

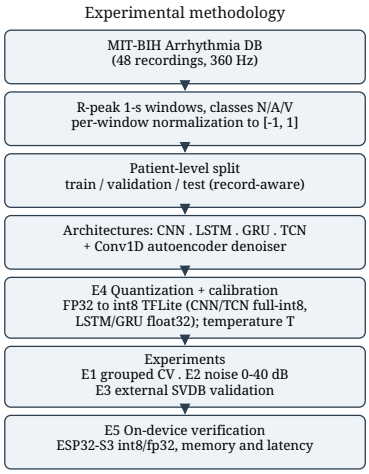

Fig. 1. Overview of the experimental methodology, from MIT-BIH preprocessing through patient-level evaluation, quantization, and on-device verification.

C. Models

Four architectures share one head (global pooling, dense 64, dropout 0.5, softmax) and schedule (Adam [23], 1e-3, batch 64, early stopping): CNN (three Conv1D blocks, 32/64/128, BN [24], kernel 5), LSTM [25] and GRU (one Conv1D + 64 recurrent), and TCN (two dilated causal blocks [26]). A Conv1D autoencoder denoiser (16/8 filters, kernel 15) is trained on clean-to-noisy reconstruction at seven SNR levels but evaluated as a rejected alternative: the recommended deployment uses a $\sim$5 ms Butterworth (0 KB) in its place (§III). CNN/TCN export to full-int8 TFLite; the evaluated Keras 3 LSTM/GRU graphs could not be converted to full-int8 through the tested converter path (TensorListStack $\rightarrow$ SELECT_TF_OPS fallback) and remain float32 under this flow. For timing we re-export both RNNs via fused UNIDIRECTIONAL_SEQUENCE_LSTM/GRU builtins (weights unchanged; Table VII). Temperature scaling (single T, NLL on validation, PC-side) calibrates confidence.

$$\sigma_n = \sigma_s \cdot 10^{-\text{SNR}/20}, \qquad \tilde{x} = x + n, \quad n \sim \mathcal{N}(0, \sigma_n^2) \tag{3}$$

Amplitude scaling for noise follows the SNR relation; waveform generation differs by artifact (baseline wander, motion transients, 50-Hz PLI, band-limited EMG, mixed; see `noise_pipeline.py`). For AWGN the standard deviation is fixed from the desired SNR, giving $\tilde{x}$ used to train the denoiser. Classifier and denoiser minimise

$$\mathcal{L}_{\text{den}} = \frac{1}{N}\sum_{i=1}^{N} \|x_i - g(\tilde{x}_i)\|_2^2, \qquad \mathcal{L}_{\text{cls}} = -\frac{1}{N}\sum_{i=1}^{N} \log p_{\tilde{y}_i}(x_i) \tag{4}$$

$$q = \text{clip}\big(\text{round}(r/s) + z,\ q_{\min}, q_{\max}\big), \qquad r \approx s\,(q - z) \tag{5}$$

The affine int8 map (5) is applied identically in the TFLite converter and the firmware's manual input conversion, so on-device tensors match the offline converter exactly.

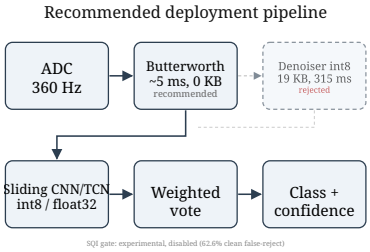

Fig. 2. Recommended pipeline: ADC → Butterworth (∼5 ms, 0 KB) → sliding CNN/TCN → weighted vote. SQI and denoiser are rejected alternatives.

$$p_k = \frac{\exp\left(z_k/T\right)}{\sum_j \exp\left(z_j/T\right)}, \qquad \mathrm{ECE} = \sum_{m=1}^{M} \frac{|B_m|}{N} \left|\mathrm{acc}_m - \mathrm{conf}_m\right|. \tag{6}$$

D. Hardware and deployment

The target is an ESP32-S3 (N16R8: 16 MB flash, 8 MB PSRAM). ADC sampling uses DMA at 36 kHz, decimated to 360 Hz. A 10-s buffer feeds 1-s windows through a Butterworth front-end (recommended; denoiser evaluated but rejected) and classifier with 50% overlap and confidence-weighted voting (full softmax summed). Memory, execution and per-stage latency are measured on silicon (BENCHMARK_MODE). Firmware acquires a 10-s buffer then classifies 19 windows back-to-back. For streaming, a new window is ready every 0.5 s (50% overlap), so real-time feasibility is assessed against the 500-ms hop interval rather than the 1-s window duration.

$$S_c = \sum_w p_c^{(w)}, \qquad \hat{y} = \arg\max_c S_c, \qquad \mathrm{conf} = \frac{\max_c S_c}{\sum_c S_c} \tag{7}$$

Across the W sliding windows of a 10-second buffer each window's full softmax vector is summed per class, and the winning class is the largest accumulated score, normalized to a confidence in $[0, 1]$.

IV. Experiments and Results

Figure 2 shows the recommended pipeline (Butterworth front-end; denoiser and SQI evaluated but disabled). Exp. 1: grouped CV (5×2, Table II); Exp. 2: noise robustness (SNR 0–40 dB, 5 artifacts × 3 front-ends, Table IV, Fig. 6); Exp. 3: partial SVDB external (Table VI); Exp. 4: paired FP32→INT8 (Table V; Table VII fused sizes); Exp. 5: S3 memory/latency (Table VII, Fig. 5), calibration (Figs. 3–4) and SQI (Supplementary Fig. S1). Exp. 2/4/calibration share one record-level split (918 windows: 493/24/401); Exp. 1 is the only 5×2.

Grouped CV (Table II, 5×2 identical folds, n=10 per model) shows CNN 0.619 ± 0.242 [95% CI 0.469–0.769] and TCN 0.615 ± 0.229 [0.473–0.758], paired $t(9)$=0.26, p=0.80 (Wilcoxon p=0.56, paired Cohen's d_z=0.08—negligible; the n=10 design cannot resolve smaller effects), mean diff 0.004 ± 0.043—no superiority. LSTM 0.582 ± 0.096 [0.522–0.642] (d_z=0.20, p=0.55), GRU 0.526 ± 0.170 [0.420–0.632] (d_z=0.36, p=0.28). APB F1 variance reflects small support (1–502 APB windows per fold; two folds ≤2 windows where F1 is undefined/0 and the mean-of-folds is unstable); pooled-confusion-matrix macro across all 10 folds is 0.643 for both CNN and TCN vs. mean-of-folds 0.619/0.615, confirming no rank change but the per-fold APB mean should not be read as a population estimate. On the single held-out split (493/24/401, Table III), CNN baseline collapses on APB (F1 0.000); weighting lifts it to 0.732 (P 0.882, R 0.625). TCN baseline already

TABLE II
Grouped CV (patient-independent 5×2)

Model	Normal	APB	PVC	Macro
CNN	0.841 ± 0.145	0.283 ± 0.359	0.733 ± 0.369	0.619 ± 0.242
TCN	0.844 ± 0.147	0.257 ± 0.319	0.746 ± 0.368	0.615 ± 0.229
LSTM	0.792 ± 0.108	0.158 ± 0.221	0.796 ± 0.098	0.582 ± 0.096
GRU	0.728 ± 0.210	0.140 ± 0.203	0.709 ± 0.279	0.526 ± 0.170

TABLE III
APB ablation (illustrative, single split)

Model	Strategy	APB Prec	APB Rec	APB F1	Macro
CNN	baseline	0.000	0.000	0.000	0.233
CNN	weighted	0.882	0.625	**0.732**	0.838
CNN	focal	0.425	0.708	0.531	0.818
CNN	balanced	0.929	0.542	0.684	**0.876**
TCN	baseline	0.875	0.583	0.700	0.753
TCN	weighted	0.750	0.625	0.682	0.876
TCN	focal	0.778	0.583	0.667	0.872
TCN	balanced	0.778	0.583	0.667	0.854
LSTM	baseline	0.143	0.042	0.065	0.299
LSTM	weighted	0.000	0.000	0.000	0.486
LSTM	focal	0.200	0.208	0.204	0.675
LSTM	balanced	0.019	0.042	0.026	0.305
GRU	baseline	0.000	0.000	0.000	0.237
GRU	weighted	0.200	0.375	0.261	0.642
GRU	focal	0.035	0.708	0.067	0.138
GRU	balanced	0.063	0.125	0.083	0.417

TABLE IV
Noise robustness (macro F1 vs SNR, averaged over the 5 artifacts of Fig. 6)

SNR (dB)	Raw	Bandpass filter	Autoencoder
0	0.233	0.455	0.475
5	0.233	0.628	0.563
10	0.286	0.726	0.586
15	0.462	0.751	0.614
20	0.683	0.723	0.629
30	0.731	0.761	0.628
40	0.736	0.751	0.629

TABLE V
Paired FP32→INT8 (5×2 folds)

Model	Float32	Int8	Δ	Disagree	Size (KB)
CNN	0.619	0.409	−0.210 ± 0.164	33.1%	77.9
TCN	0.615	0.378	−0.238 ± 0.165	39.7%	70.1
LSTM	0.582	—	—	—	130.3
GRU	0.526	—	—	—	107.5

scores APB 0.700, within rounding of CNN's best mitigated 0.732. Table III is an illustrative single-split ceiling (only 24 APB windows), not a generalization estimate. Under the same 5×2 protocol, weighted vs baseline gives ΔAPB -0.005 (p=0.53) / $+0.011$ (p=0.53) and Δmacro -0.008 (p=0.33) / -0.039 (p=0.32), confirming no improvement; Table II's 0.619 is the honest number. Partial SVDB external (Table VI, 29,554 windows, APB absent) gives N/V-macro 0.562; 3-class 0.375 is mechanically biased. Paired INT8 on identical folds (Table V; PTQ representative set: 200 train-split windows drawn uniformly at random, preserving the natural class imbalance; full-int8, TFLITE_BUILTINS_INT8) degrades macro by 0.210 ± 0.164 (CNN, 33% disagree) and 0.238 ± 0.165 (TCN, 40% disagree); the evaluated LSTM/GRU graphs remain float32 through the tested path. On the same CNN folds, QAT recovers to 0.584 ± 0.188 (PTQ 0.549 ± 0.178 in that run; $\Delta_{\text{QAT}}-\text{PTQ}+0.036 \pm 0.085$, p=0.42, 38% of PTQ loss, 16.1% vs FP32) but not to FP32. Table V LSTM/GRU sizes are the pre-fusion SELECT_TF_OPS exports; Table VII lists the smaller fused exports (§III-C) used for on-device timing. Degradation is specific to this pipeline and not generalized. On the designated held-out split, temperature scaling (T=0.350 fit on validation, test n=918) gives ECE 0.328 $\rightarrow$ 0.051, NLL 0.700 $\rightarrow$ 0.407, Brier 0.381 $\rightarrow$ 0.210 (Figs. 3–4). The fitted T=0.350<1 indicates under-confident logits on the designated validation split; temperature scaling therefore sharpened rather than softened the predictive distribution. The same PTQ INT8 model on the identical split yields ECE 0.481 $\rightarrow$ 0.352, NLL 4.56 $\rightarrow$ 1.11, Brier 0.94 $\rightarrow$ 0.60 with a separate T_{INT8}=5.0 (upper bound of the search) fitted on INT8 validation outputs (Table VIII); FP32 calibration does not transfer, and deployed INT8 confidences remain substantially miscalibrated (44% argmax agreement with FP32). QAT was attempted (TF-MOT) but Conv1D is not supported by the default 8-bit path; PTQ therefore represents the deployed pipeline.

TABLE VI
EXTERNAL VALIDATION (SVDB, N/V ONLY)

Class	F1	Windows
Normal	0.663	28,470
APB	—	0
PVC	0.462	1,084
N/V-macro	0.562	29,554
3-class (biased)	0.375	29,554

TABLE VII
DEPLOYMENT ON ESP-NN (PER-WINDOW LATENCY) — DENOISER (REJECTED) VS. RECOMMENDED BUTTERWORTH

Model	Macro	Size (KB)	Denoiser (ms)	Recomm. (ms)	RTF_{rec}	ΔINT8
CNN	0.619	77.9	500	189	0.38	-0.210
TCN	0.615	70.1	609	298	0.60	-0.238
LSTM	0.582	126.5	1290	980	1.96	—
GRU	0.526	103.7	1091	781	1.56	—

A. *On-Device Feasibility Verification*

The quantized CNN/TCN occupy 77.9/70.1 KB plus the 19 KB denoiser in flash; the float32 LSTM/GRU occupy

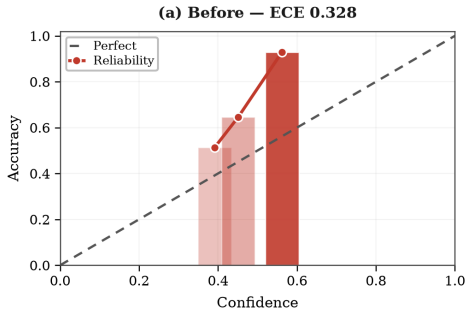

Fig. 3. Calibration (a) Before — ECE 0.328 (robust CNN, T=0.350 on validation, held-out n=918).

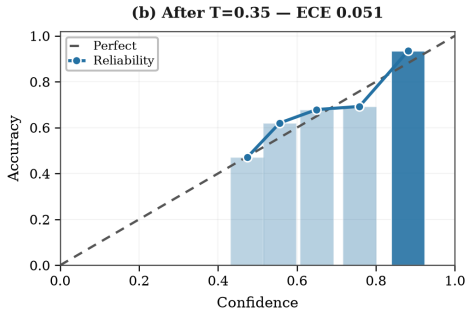

Fig. 4. Calibration (b) After — ECE 0.051, NLL 0.700→0.407, Brier 0.381→0.210 — 10 bins, opacity = count.

126.5/103.7 KB. The tensor arena is 300 KB internal SRAM, split 96 KB denoiser / 204 KB classifier to accommodate ESP-NN scratch; leaving $\approx$ 49 KB heap by budget (Table VII, Fig. 5). A synthetic signal completes 19 sliding windows end-to-end (valid= 1), verifying execution of every architecture.

Measured per-window latency on *ESP-NN-optimized kernels* (BENCHMARK_MODE, n = 19, 240 MHz, fixed-seed input, three runs, $< 0.1\%$): CNN 500 ms (315 denoise + 184 classify), TCN 609 ms (315 + 293), LSTM 1290 ms, GRU 1091 ms. Against the 0.5-s hop, denoiser-in-loop is marginal (RTF 1.00/1.22, 95% duty cycle). Replacing the denoiser with the $\sim$5 ms Butterworth gives per-window classifier+filter latency of 189/298 ms (184/293 classify + $\sim$5 filter; RTF 0.38/0.60, 36% duty cycle), so CNN/TCN meet real-time precisely because the denoiser must go; float32 LSTM/GRU fail (≥ 1.55). For a fixed input, each int8 output tensor element on S3 matched the PC-side TFLite tensor exactly; portable reference kernels had run $7.6\times$/$5.9\times$ slower, inverting the ranking. ESP-NN accelerates int8 conv $\approx 17\times$ vs $\sim$20% for float32 sequence. Notably, esp-nn v1.0.0's S3 assembly kernels produced silently incorrect conv outputs on our shapes; our harness localized the fault and upstream master fixes resolve it. All final results use the corrected kernels; Table V quantization metrics are PC-side and independent of this fault. DAC$\rightarrow$ADC replay was not performed because S3 lacks an integrated DAC (external DAC required, left for future hardware validation).

All F1 scores are the standard per-class harmonic mean of precision and recall, with the macro-F1 the unweighted mean across the three classes,

TABLE VIII
INT8 CALIBRATION (SAME 918-WINDOW TEST SPLIT)

	ECE	NLL	Brier
FP32 raw	0.328	0.700	0.381
FP32 T=0.35	0.051	0.407	0.210
INT8 raw	0.481	4.56	0.94
INT8 T_{INT8}=5.0	0.352	1.11	0.60

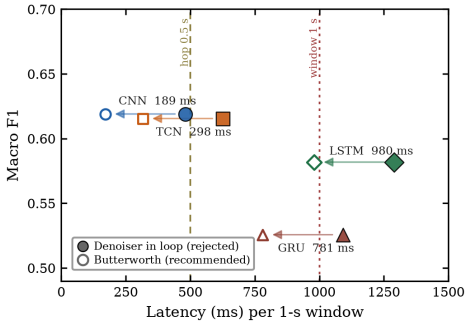

Fig. 5. Pareto: macro vs. latency. Filled = denoiser (rejected); hollow + arrow = recommended Butterworth (189/298 ms CNN/TCN, 980/781 ms LSTM/GRU). Bubble = flash size; colors = architecture. Budgets: 1-s window, 0.5-s hop.

$$F1_c = \frac{2\,\mathrm{TP}_c}{2\,\mathrm{TP}_c + \mathrm{FP}_c + \mathrm{FN}_c}, \qquad \text{Macro-F1} = \frac{1}{C}\sum_{c=1}^{C} F1_c \tag{8}$$

V. Discussion and Limitations

Single-lead limits scope (no ST localization/axis). APB is beat-level, not AF (rhythm-level, scoped out). On-device we report on-device computational feasibility verification (19 windows, numerical equivalence and timing are verified on silicon, while analog-front-end acquisition-domain classification remains future work) and measured latency (Table VII); DAC→ADC replay was not performed because S3 lacks an integrated DAC (external DAC required) and real AD8232 acquisition and battery life remain future hardware/ethics work.

Grouped CV (5×2, Table II, n=10) ties CNN 0.619 ± 0.242 [0.469–0.769] and TCN 0.615 ± 0.229 [0.473–0.758] (paired $t(9)$=0.26, p=0.80; no superiority), LSTM 0.582 ± 0.096, GRU 0.526 ± 0.170; large SD reflects fold composition and APB support 1–502 per fold (pooled macro 0.643 for both CNN/TCN). Noise: Butterworth wins $3/5$ artifacts, raw wins motion, AE only low-SNR PLI (Fig. 6); 19 KB + 315 ms buys <0.1 macro. The autoencoder is an evaluated-and-rejected alternative; learned preprocessing is not automatically advantageous in TinyML when marginal benefit is small relative to latency and memory cost. SQI gate rejects 62.6% clean vs. 27.9% corrupted at 0.35 (Supplementary Fig. S1) — experimental and disabled in the recommended pipeline. Simple amplitude/spectral thresholds did not reliably distinguish valid morphological variability from corruption; future SQI should be learned or calibrated using explicitly labeled signal-quality data. Quantization (Table V) costs 0.21–0.24

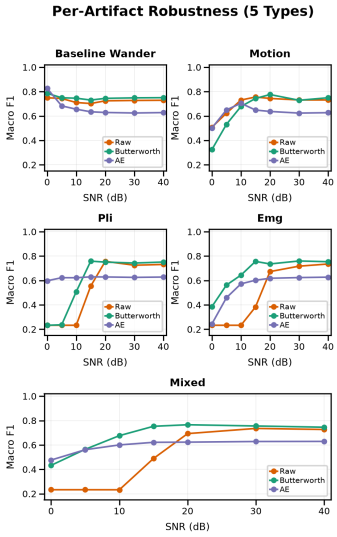

Fig. 6. Per-artifact robustness. Butterworth wins 3/5, raw wins motion, AE only wins PLI low-SNR — denoiser not justified.

macro with 33/40% disagreement (PTQ only; QAT attempted but Conv1D not supported by default 8-bit path), arguing for calibrated abstention. Class weighting does not rescue APB under grouped CV (CNN ΔAPB -0.005 ± 0.015 p=0.53; TCN $+0.011\pm0.033$ p=0.53; Δmacro $-0.008/-0.039$, p=0.33/0.32).

Deployment (Table VII) on ESP-NN: denoiser-in-loop RTF 1.00/1.22 (marginal), Butterworth RTF 0.38/0.60 (real-time); evaluated LSTM/GRU remain float32 and above budget. Memory is not the limiter (PSRAM unused). The esp-nn v1.0.0 numerical fault (silent conv corruption, fixed upstream; all final results use corrected kernels) argues for probability-level on-device validation when swapping kernels. Recommended: ECG @360 Hz → Butterworth (∼5 ms) → 1-s windows (50% overlap) → CNN/TCN → weighted vote → class+confidence; denoiser/SQI remain rejected.

Earlier MIT-BIH studies often report substantially higher performance (e.g., ≥ 0.90 [6], [11], [7]) under patient-specific or non-patient-independent evaluation protocols. Direct numerical comparison with the present patient-disjoint results (0.53–0.62) is therefore inappropriate.

Reproducibility: PhysioNet pull [21], [1], [27], frozen 5×2 folds (`results/folds_5x2.json`), 96 ckpts (`results/group_kfold_ckpt/` and `results/paired_quant/`), seeds 0,1 documented; TensorFlow 2.21 / Keras 3 / scikit-learn [28]; firmware header conversion and BENCHMARK_MODE share the tree; all tables/figs regenerate end-to-end. ESP-NN is an in-tree build; LSTM/GRU timings use fused float32 exports (weights unchanged). Source code, frozen folds, checkpoints, and firmware are publicly available at https://github.com/touhidsiddiqueeraj-bit/heartlens.

VI. Conclusion

We presented a comparative evaluation of CNN, LSTM, GRU, and TCN for edge ECG screening on a single patient-

level MIT-BIH pipeline, with grouped CV, noise robustness, partial SVDB external validation, paired INT8 impact, calibration, and measured ESP32-S3 execution. Under the evaluated configuration, CNN and TCN meet the streaming hop budget (RTF 0.38/0.60) once the learned denoiser is replaced by a Butterworth filter—from cost preference to hard requirement—with outputs matching PC reference (bit-exact per tensor element). The evaluated LSTM/GRU graphs remained float32 through the tested converter path and exceed the budget: kernel-aware choice is favored under this deployment configuration, not universally settled.

Use of a Large Language Model (LLM) in the Manuscript

The authors hereby declare that the use of ChatGPT or any LLM in the preparation of this research paper is either not applicable, or if utilized, has been duly acknowledged within the paper (preferably in the 'ACKNOWLEDGEMENT' Section), where it is explicitly stated that such models were employed solely for non-intellectual purposes. The authors confirm that the output generated by ChatGPT or any LLM has been carefully reviewed and approved by them. The responsibility for the content and accuracy of the information rests solely with the authors.

References

[1] G. B. Moody and R. G. Mark, "The impact of the MIT-BIH arrhythmia database," *IEEE Engineering in Medicine and Biology Magazine*, vol. 20, no. 3, pp. 45–50, 2001. [Online]. Available: https://doi.org/10.1109/51.932724

[2] P. de Chazal, M. O'Dwyer, and R. B. Reilly, "Automatic classification of heartbeats using ECG morphology and heartbeat interval features," *IEEE Transactions on Biomedical Engineering*, vol. 51, no. 7, pp. 1196–1206, 2004. [Online]. Available: https://doi.org/10.1109/TBME.2004.827359

[3] R. Davidson, A. Natarajan *et al.*, "TensorFlow Lite Micro: Embedded machine learning for TinyML systems," *Proceedings of Machine Learning and Systems*, vol. 3, pp. 800–811, 2021. [Online]. Available: https://arxiv.org/abs/2010.08678

[4] F. Guede-Fernandez *et al.*, "Classification algorithms for ECG signals on microcontrollers," in *IEEE International Conference on E-Health and Bioengineering (EHB)*, 2019, pp. 1–4. [Online]. Available: https://doi.org/10.1109/EHB47216.2019.8969893

[5] G. B. Moody and R. G. Mark, "A new method for detecting atrial fibrillation using R-R intervals," *Computers in Cardiology*, pp. 227–230, 1983.

[6] S. Kiranyaz, T. Ince, and M. Gabbouj, "Real-time patient-specific ECG classification by 1-D convolutional neural networks," *IEEE Transactions on Biomedical Engineering*, vol. 63, no. 3, pp. 664–675, 2016. [Online]. Available: https://doi.org/10.1109/TBME.2015.2468589

[7] A. Y. Hannun, P. Rajpurkar, M. Haghpanahi *et al.*, "Cardiologist-level arrhythmia detection and classification in ambulatory electrocardiograms using a deep neural network," *Nature Medicine*, vol. 25, pp. 65–69, 2019. [Online]. Available: https://doi.org/10.1038/s41591-018-0268-3

[8] P. Rajpurkar, A. Y. Hannun, M. Haghpanahi, C. Bourn, and A. Y. Ng, "Cardiologist-level arrhythmia detection with convolutional neural networks," arXiv:1707.01836, 2017. [Online]. Available: https://arxiv.org/abs/1707.01836

[9] A. H. Ribeiro, M. H. Ribeiro, G. M. M. Paixão *et al.*, "Automatic diagnosis of the 12-lead ECG using a deep neural network," *Nature Communications*, vol. 11, p. 1760, 2020. [Online]. Available: https://doi.org/10.1038/s41467-020-15432-4

[10] E. J. d. S. Luz, W. R. Schwartz, G. Cámara-Chávez, and D. Menotti, "ECG-based heartbeat classification for arrhythmia detection: A survey," *Computer Methods and Programs in Biomedicine*, vol. 127, pp. 144–164, 2016. [Online]. Available: https://doi.org/10.1016/j.cmpb.2015.12.008

[11] U. R. Acharya, S. L. Oh, Y. Hagiwara, J. H. Tan, M. Adam, A. Gertych, and R. S. Tan, "A deep convolutional neural network model to classify heartbeats," *Computers in Biology and Medicine*, vol. 89, pp. 389–396, 2017. [Online]. Available: https://doi.org/10.1016/j.compbiomed.2017.08.022

[12] O. Yildirim, P. Pławiak, R.-S. Tan, and U. R. Acharya, "Arrhythmia detection using deep convolutional neural network with long duration ECG signals," *Computers in Biology and Medicine*, vol. 102, pp. 411–420, 2018. [Online]. Available: https://doi.org/10.1016/j.compbiomed.2018.09.009

[13] M. A. Serhani, H. T. El Kassabi, H. Ismail, and A. N. Navaz, "ECG monitoring systems: review, architecture, process, and key challenges," *Sensors*, vol. 20, no. 6, p. 1796, 2020. [Online]. Available: https://doi.org/10.3390/s20061796

[14] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 2704–2713. [Online]. Available: https://doi.org/10.1109/CVPR.2018.00286

[15] R. Krishnamoorthi, "Quantizing deep convolutional networks for efficient inference: insights and empirical results," arXiv:1806.08342, 2018. [Online]. Available: https://arxiv.org/abs/1806.08342

[16] P. Warden and D. Situnayake, *TinyML: Machine learning with TensorFlow Lite on Arduino and ultra-low-power microcontrollers*. O'Reilly Media, 2020. [Online]. Available: https://tinymlbook.com/

[17] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *International Conference on Machine Learning (ICML)*, 2017, pp. 1321–1330. [Online]. Available: https://proceedings.mlr.press/v70/guo17a.html

[18] M. P. Naeini, G. F. Cooper, and M. Hauskrecht, "Obtaining well calibrated probabilities using bayesian binning then quantile calibration," in *AAAI Conference on Artificial Intelligence*, 2015, pp. 2901–2907. [Online]. Available: https://doi.org/10.1609/aaai.v29i1.9602

[19] Espressif Systems, "ESP32-S3 technical reference manual," Espressif Systems documentation, 2023. [Online]. Available: https://docs.espressif.com/projects/esp-idf/en/latest/esp32s3/

[20] G. B. Moody and R. G. Mark, "The MIT-BIH arrhythmia database on CD-ROM and software for use with it," *Computers in Cardiology*, pp. 185–188, 1990. [Online]. Available: https://physionet.org/content/mitdb/

[21] A. L. Goldberger *et al.*, "PhysioBank, PhysioToolkit, and PhysioNet: Components of a new research resource for complex physiologic signals," *Circulation*, vol. 101, no. 23, pp. e215–e220, 2000. [Online]. Available: https://doi.org/10.1161/01.CIR.101.23.e215

[22] G. D. Clifford, F. Azuaje, and P. McSharry, Eds., *Advanced Methods and Tools for ECG Data Analysis*. Artech House, 2006.

[23] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," arXiv:1412.6980, 2015. [Online]. Available: https://arxiv.org/abs/1412.6980

[24] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," arXiv:1502.03167, 2015. [Online]. Available: https://arxiv.org/abs/1502.03167

[25] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997. [Online]. Available: https://doi.org/10.1162/neco.1997.9.8.1735

[26] S. Bai, J. Z. Kolter, and V. Koltun, "An empirical evaluation of generic convolutional and recurrent networks for sequence modeling," arXiv:1803.01271, 2018.

[27] A. L. Goldberger *et al.*, "MIT-BIH supraventricular arrhythmia database (SVDB)," PhysioNet, 2010.

[28] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.